step2_advanced_validation

May 27, 2026

1 Step 2: Robust Validation, Sampling, and Ensembling

This notebook expands evaluation metrics, applies sampling strategies on the training split only, and blends multiple tree models for more stable PR-AUC on imbalanced data.

[1]:
```
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.feature_selection import mutual_info_classif
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split

RANDOM_STATE = 42
DATA_PATH = Path('DataSet.csv')
TARGET_COL = 'F3924'
ID_COL = 'Unnamed: 0'
LEAKY_FEATURES = ['F3912']

BANK_FEATURES = [
    'F115', 'F321', 'F527', 'F531', 'F670', 'F1692', 'F2082', 'F2122',
    'F2582', 'F2678', 'F2737', 'F2956', 'F3043', 'F3836', 'F3887',
    'F3889', 'F3891', 'F3894',
]

PLACEHOLDER_VALUES = {-99999999, 99999999, -9999999, 9999999, -999999, 999999,␣
↪-9999, 9999}
LARGE_ABS_THRESHOLD = 1e7
PLACEHOLDER_MIN_FRAC = 0.002
```

```
TOP_MI = 25
TOP_GAP = 25
```

[2]:
```
df = pd.read_csv(DATA_PATH)
print(f'Loaded shape: {df.shape}')

if ID_COL in df.columns:
    df = df.drop(columns=[ID_COL])

y = df[TARGET_COL].astype(int)
X = df.drop(columns=[TARGET_COL], errors='ignore')

if LEAKY_FEATURES:
    X = X.drop(columns=[col for col in LEAKY_FEATURES if col in X.columns],␣
 ↪errors='ignore')

X = X.replace(list(PLACEHOLDER_VALUES), np.nan)
X = X.replace([np.inf, -np.inf], np.nan)
X = X.apply(pd.to_numeric, errors='coerce')

def detect_placeholder_values(frame: pd.DataFrame, abs_threshold: float,␣
 ↪min_frac: float) -> dict[str, float]:
    placeholder_map: dict[str, float] = {}
    for col in frame.columns:
        series = frame[col].dropna()
        if series.empty:
            continue
        extreme = series[series.abs() >= abs_threshold]
        if extreme.empty:
            continue
        counts = extreme.value_counts()
        candidate = counts.index[0]
        if counts.iloc[0] / len(series) >= min_frac:
            placeholder_map[col] = candidate
    return placeholder_map

placeholder_map = detect_placeholder_values(X, LARGE_ABS_THRESHOLD,␣
 ↪PLACEHOLDER_MIN_FRAC)
for col, value in placeholder_map.items():
    X[col] = X[col].replace(value, np.nan)

print('Features after cleaning:', X.shape)
print('Placeholder columns detected:', len(placeholder_map))
print('Target base rate:', y.mean())
```

```
Loaded shape: (9082, 3925)
Features after cleaning: (9082, 3922)
Placeholder columns detected: 76
Target base rate: 0.008918740365558247
```

[3]:
```python
X_train_raw, X_valid_raw, y_train, y_valid = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=RANDOM_STATE,
)

print('Train shape:', X_train_raw.shape)
print('Valid shape:', X_valid_raw.shape)
```

```
Train shape: (7265, 3922)
Valid shape: (1817, 3922)
```

[4]:
```python
def build_row_stats(frame: pd.DataFrame) -> pd.DataFrame:
    values = frame.to_numpy(dtype=float)
    mask = ~np.isnan(values)
    non_missing = mask.sum(axis=1)
    total = values.shape[1]
    missing_rate = 1.0 - (non_missing / total)

    zero_rate = np.where(non_missing > 0, (values == 0).sum(axis=1) /␣
 ↪non_missing, 0)
    positive_rate = np.where(non_missing > 0, (values > 0).sum(axis=1) /␣
 ↪non_missing, 0)
    negative_rate = np.where(non_missing > 0, (values < 0).sum(axis=1) /␣
 ↪non_missing, 0)

    with np.errstate(all='ignore'):
        mean = np.nanmean(values, axis=1)
        std = np.nanstd(values, axis=1)
        min_val = np.nanmin(values, axis=1)
        max_val = np.nanmax(values, axis=1)
        median = np.nanmedian(values, axis=1)
        q25 = np.nanpercentile(values, 25, axis=1)
        q75 = np.nanpercentile(values, 75, axis=1)
        abs_mean = np.nanmean(np.abs(values), axis=1)

    iqr = q75 - q25

    return pd.DataFrame({
        'row_non_missing_count': non_missing,
        'row_missing_rate': missing_rate,
```

```
        'row_zero_rate': zero_rate,
        'row_positive_rate': positive_rate,
        'row_negative_rate': negative_rate,
        'row_mean': mean,
        'row_std': std,
        'row_min': min_val,
        'row_max': max_val,
        'row_median': median,
        'row_q25': q25,
        'row_q75': q75,
        'row_iqr': iqr,
        'row_abs_mean': abs_mean,
    }, index=frame.index)

row_stats_train = build_row_stats(X_train_raw)
row_stats_valid = build_row_stats(X_valid_raw)

print('Row stats shape (train):', row_stats_train.shape)
```

```
Row stats shape (train): (7265, 14)
```

[5]:
```
bank_features = [col for col in BANK_FEATURES if col in X_train_raw.columns and␣
 ↪X_train_raw[col].notna().any()]

mi_candidates = X_train_raw.loc[:, X_train_raw.nunique(dropna=True) > 1]
top_mi_cols = []
if mi_candidates.shape[1] > 0:
    mi_imputer = SimpleImputer(strategy='median')
    mi_values = mi_imputer.fit_transform(mi_candidates)
    mi_scores = mutual_info_classif(mi_values, y_train,␣
 ↪random_state=RANDOM_STATE)
    mi_series = pd.Series(mi_scores, index=mi_candidates.columns).
 ↪sort_values(ascending=False)
    top_mi_cols = mi_series.head(TOP_MI).index.tolist()

missing_pos = X_train_raw.loc[y_train == 1].isna().mean()
missing_neg = X_train_raw.loc[y_train == 0].isna().mean()
missing_gap = (missing_pos - missing_neg).abs().sort_values(ascending=False)
top_gap_cols = missing_gap.head(TOP_GAP).index.tolist()

selected_cols = []
for col in bank_features + top_mi_cols + top_gap_cols:
    if col not in selected_cols:
        selected_cols.append(col)

missing_flags_train = X_train_raw[top_gap_cols].isna().astype(int).
 ↪add_prefix('miss_')
```

```
missing_flags_valid = X_valid_raw[top_gap_cols].isna().astype(int).
 ↪add_prefix('miss_')

X_train_compact = pd.concat([X_train_raw[selected_cols], row_stats_train,␣
 ↪missing_flags_train], axis=1)
X_valid_compact = pd.concat([X_valid_raw[selected_cols], row_stats_valid,␣
 ↪missing_flags_valid], axis=1)

all_missing_cols = X_train_compact.columns[X_train_compact.isna().mean() == 1.0]
X_train_compact = X_train_compact.drop(columns=all_missing_cols)
X_valid_compact = X_valid_compact.drop(columns=all_missing_cols)
X_valid_compact = X_valid_compact[X_train_compact.columns]

print('Bank features kept:', len(bank_features))
print('Top MI columns:', len(top_mi_cols))
print('Top missingness-gap columns:', len(top_gap_cols))
print('Compact train shape:', X_train_compact.shape)
print('Compact valid shape:', X_valid_compact.shape)
```

```
Bank features kept: 16
Top MI columns: 25
Top missingness-gap columns: 25
Compact train shape: (7265, 105)
Compact valid shape: (1817, 105)
```

[6]:
```
def report_metrics(name: str, y_true: pd.Series, y_prob: np.ndarray, threshold:␣
 ↪float = 0.5) -> dict:
    pr_auc = average_precision_score(y_true, y_prob)
    roc_auc = roc_auc_score(y_true, y_prob)
    y_pred = (y_prob >= threshold).astype(int)
    macro_f1 = f1_score(y_true, y_pred, average='macro')
    minority_f1 = f1_score(y_true, y_pred, pos_label=1)
    balanced_acc = balanced_accuracy_score(y_true, y_pred)
    cm = confusion_matrix(y_true, y_pred)

    print(f'=== {name} Performance ===')
    print(f'PR-AUC:          {pr_auc:.4f}')
    print(f'ROC-AUC:         {roc_auc:.4f}')
    print(f'Macro F1-Score: {macro_f1:.4f}')
    print(f'Minority F1:     {minority_f1:.4f}')
    print(f'Balanced Acc:    {balanced_acc:.4f}')
    print('')
    print('Confusion Matrix:')
    print(cm)
    print('')
    print('Detailed Report:')
    print(classification_report(y_true, y_pred, digits=4))
```

```
    print('=' * 30)
    print('')

    return {
        'pr_auc': pr_auc,
        'roc_auc': roc_auc,
        'macro_f1': macro_f1,
        'minority_f1': minority_f1,
        'balanced_acc': balanced_acc,
    }
```

1.1 Phase 1: Baseline Models (No Sampling)

[7]:
```
try:
    import xgboost as xgb
except ImportError:
    xgb = None
    print('xgboost not installed. Install with: pip install xgboost')

try:
    import lightgbm as lgb
except ImportError:
    lgb = None
    print('lightgbm not installed. Install with: pip install lightgbm')

def build_xgb(scale_pos_weight: float):
    return xgb.XGBClassifier(
        n_estimators=500,
        max_depth=5,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric='aucpr',
        scale_pos_weight=scale_pos_weight,
        tree_method='hist',
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )

def build_lgb(scale_pos_weight: float):
    return lgb.LGBMClassifier(
        n_estimators=500,
        learning_rate=0.05,
        num_leaves=31,
        subsample=0.8,
        colsample_bytree=0.8,
        objective='binary',
```

```
        scale_pos_weight=scale_pos_weight,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        verbosity=-1,
    )

def fit_and_report(name: str, model, X_train, y_train, X_valid, y_valid):
    model.fit(X_train, y_train)
    y_prob = model.predict_proba(X_valid)[:, 1]
    report_metrics(name, y_valid, y_prob)
    return y_prob

base_scale_pos_weight = (y_train == 0).sum() / max((y_train == 1).sum(), 1)
xgb_prob = None
lgb_prob = None

if xgb is not None:
    xgb_prob = fit_and_report(
        'XGBoost (no sampling)',
        build_xgb(base_scale_pos_weight),
        X_train_compact,
        y_train,
        X_valid_compact,
        y_valid,
    )

if lgb is not None:
    lgb_prob = fit_and_report(
        'LightGBM (no sampling)',
        build_lgb(base_scale_pos_weight),
        X_train_compact,
        y_train,
        X_valid_compact,
        y_valid,
    )
```

```
=== XGBoost (no sampling) Performance ===
PR-AUC:         0.8968
ROC-AUC:        0.9986
Macro F1-Score: 0.8861
Minority F1:    0.7742
Balanced Acc:   0.8742

Confusion Matrix:
[[1798    3]
 [   4   12]]

Detailed Report:
```

```
              precision    recall  f1-score   support

           0     0.9978    0.9983    0.9981      1801
           1     0.8000    0.7500    0.7742        16

    accuracy                         0.9961      1817
   macro avg     0.8989    0.8742    0.8861      1817
weighted avg     0.9960    0.9961    0.9961      1817

==============================

=== LightGBM (no sampling) Performance ===
PR-AUC:          0.8106
ROC-AUC:         0.9912
Macro F1-Score: 0.9131
Minority F1:    0.8276
Balanced Acc:   0.8747

Confusion Matrix:
[[1800    1]
 [   4   12]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9978    0.9994    0.9986      1801
           1     0.9231    0.7500    0.8276        16

    accuracy                         0.9972      1817
   macro avg     0.9604    0.8747    0.9131      1817
weighted avg     0.9971    0.9972    0.9971      1817

==============================
```

1.2 Phase 2: Sampling Strategies (Train Split Only)

[8]:
```
try:
    from imblearn.combine import SMOTETomek
    from imblearn.over_sampling import SMOTE
    from imblearn.under_sampling import RandomUnderSampler
    has_imblearn = True
except ImportError:
    has_imblearn = False
    print('imbalanced-learn not installed. Install with: pip install␣
 ↪imbalanced-learn')
```

```
X_train_smote = None
X_train_rus = None
X_train_hybrid = None
y_train_smote = None
y_train_rus = None
y_train_hybrid = None
X_train_imp = None
X_valid_imp = None

if has_imblearn:
    sampler_imputer = SimpleImputer(strategy='median')
    X_train_imp = sampler_imputer.fit_transform(X_train_compact)
    X_valid_imp = sampler_imputer.transform(X_valid_compact)

    smote = SMOTE(sampling_strategy=0.1, random_state=RANDOM_STATE)
    X_train_smote, y_train_smote = smote.fit_resample(X_train_imp, y_train)

    rus = RandomUnderSampler(sampling_strategy=0.1, random_state=RANDOM_STATE)
    X_train_rus, y_train_rus = rus.fit_resample(X_train_imp, y_train)

    smote_tomek = SMOTETomek(sampling_strategy=0.1, random_state=RANDOM_STATE)
    X_train_hybrid, y_train_hybrid = smote_tomek.fit_resample(X_train_imp,␣
 ↪y_train)

    print(f'Original training shape: {X_train_compact.shape} (Positives:␣
 ↪{y_train.sum()})')
    print(f'SMOTE training shape:    {X_train_smote.shape} (Positives:␣
 ↪{y_train_smote.sum()})')
    print(f'RUS training shape:      {X_train_rus.shape} (Positives:␣
 ↪{y_train_rus.sum()})')
    print(f'SMOTETomek shape:        {X_train_hybrid.shape} (Positives:␣
 ↪{y_train_hybrid.sum()})')
```

```
Original training shape: (7265, 105) (Positives: 65)
SMOTE training shape:    (7920, 105) (Positives: 720)
RUS training shape:      (715, 105) (Positives: 65)
SMOTETomek shape:        (7890, 105) (Positives: 705)
```

```
C:\Users\amart\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.10_qbz5n
2kfra8p0\LocalCache\local-packages\Python310\site-
packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not
find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by
setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
  File "C:\Users\amart\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.
10_qbz5n2kfra8p0\LocalCache\local-packages\Python310\site-
```

```
packages\joblib\externals\loky\backend\context.py", line 257, in
_count_physical_cores
    cpu_info = subprocess.run(
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 503, in run
    with Popen(*popenargs, **kwargs) as process:
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 971, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 1456, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
```

[9]:
```python
xgb_prob_smote = None
lgb_prob_smote = None

if has_imblearn:
    if xgb is not None:
        spw = (y_train_smote == 0).sum() / max((y_train_smote == 1).sum(), 1)
        xgb_prob_smote = fit_and_report(
            'XGBoost (SMOTE)',
            build_xgb(spw),
            X_train_smote,
            y_train_smote,
            X_valid_imp,
            y_valid,
        )
    if lgb is not None:
        spw = (y_train_smote == 0).sum() / max((y_train_smote == 1).sum(), 1)
        lgb_prob_smote = fit_and_report(
            'LightGBM (SMOTE)',
            build_lgb(spw),
            X_train_smote,
            y_train_smote,
            X_valid_imp,
            y_valid,
        )

    if xgb is not None:
        spw = (y_train_rus == 0).sum() / max((y_train_rus == 1).sum(), 1)
        fit_and_report(
            'XGBoost (RUS)',
            build_xgb(spw),
            X_train_rus,
            y_train_rus,
            X_valid_imp,
            y_valid,
```

```
        )
    if lgb is not None:
        spw = (y_train_rus == 0).sum() / max((y_train_rus == 1).sum(), 1)
        fit_and_report(
            'LightGBM (RUS)',
            build_lgb(spw),
            X_train_rus,
            y_train_rus,
            X_valid_imp,
            y_valid,
        )

    if xgb is not None:
        spw = (y_train_hybrid == 0).sum() / max((y_train_hybrid == 1).sum(), 1)
        fit_and_report(
            'XGBoost (SMOTETomek)',
            build_xgb(spw),
            X_train_hybrid,
            y_train_hybrid,
            X_valid_imp,
            y_valid,
        )
    if lgb is not None:
        spw = (y_train_hybrid == 0).sum() / max((y_train_hybrid == 1).sum(), 1)
        fit_and_report(
            'LightGBM (SMOTETomek)',
            build_lgb(spw),
            X_train_hybrid,
            y_train_hybrid,
            X_valid_imp,
            y_valid,
        )
```

```
=== XGBoost (SMOTE) Performance ===
PR-AUC:         0.9089
ROC-AUC:        0.9987
Macro F1-Score: 0.9406
Minority F1:    0.8824
Balanced Acc:   0.9679

Confusion Matrix:
[[1798    3]
 [   1   15]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9994    0.9983    0.9989      1801
```

```
           1     0.8333    0.9375    0.8824        16

    accuracy                         0.9978      1817
   macro avg     0.9164    0.9679    0.9406      1817
weighted avg     0.9980    0.9978    0.9979      1817

==============================

=== LightGBM (SMOTE) Performance ===
PR-AUC:         0.8759
ROC-AUC:        0.9988
Macro F1-Score: 0.8992
Minority F1:    0.8000
Balanced Acc:   0.8744

Confusion Matrix:
[[1799    2]
 [   4   12]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9978    0.9989    0.9983      1801
           1     0.8571    0.7500    0.8000        16

    accuracy                         0.9967      1817
   macro avg     0.9275    0.8744    0.8992      1817
weighted avg     0.9965    0.9967    0.9966      1817

==============================

=== XGBoost (RUS) Performance ===
PR-AUC:         0.6888
ROC-AUC:        0.9919
Macro F1-Score: 0.7578
Minority F1:    0.5217
Balanced Acc:   0.8700

Confusion Matrix:
[[1783   18]
 [   4   12]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9978    0.9900    0.9939      1801
           1     0.4000    0.7500    0.5217        16
```

```
    accuracy                           0.9879      1817
   macro avg     0.6989    0.8700    0.7578      1817
weighted avg     0.9925    0.9879    0.9897      1817

==============================

=== LightGBM (RUS) Performance ===
PR-AUC:         0.6933
ROC-AUC:        0.9845
Macro F1-Score: 0.7832
Minority F1:    0.5714
Balanced Acc:   0.8711

Confusion Matrix:
[[1787   14]
 [   4   12]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9978    0.9922    0.9950      1801
           1     0.4615    0.7500    0.5714        16

    accuracy                           0.9901      1817
   macro avg     0.7297    0.8711    0.7832      1817
weighted avg     0.9930    0.9901    0.9913      1817

==============================

=== XGBoost (SMOTETomek) Performance ===
PR-AUC:         0.9061
ROC-AUC:        0.9988
Macro F1-Score: 0.9406
Minority F1:    0.8824
Balanced Acc:   0.9679

Confusion Matrix:
[[1798    3]
 [   1   15]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9994    0.9983    0.9989      1801
           1     0.8333    0.9375    0.8824        16

    accuracy                           0.9978      1817
   macro avg     0.9164    0.9679    0.9406      1817
```

```
weighted avg     0.9980    0.9978    0.9979      1817

==============================

=== LightGBM (SMOTETomek) Performance ===
PR-AUC:          0.8589
ROC-AUC:         0.9984
Macro F1-Score: 0.9328
Minority F1:     0.8667
Balanced Acc:    0.9060

Confusion Matrix:
[[1800    1]
 [   3   13]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9983    0.9994    0.9989      1801
           1     0.9286    0.8125    0.8667        16

    accuracy                         0.9978      1817
   macro avg     0.9635    0.9060    0.9328      1817
weighted avg     0.9977    0.9978    0.9977      1817

==============================
```

1.3 Phase 3: Weighted Blending Ensemble

[10]:
```python
best_prob = None

if xgb_prob_smote is not None and lgb_prob_smote is not None:
    blend_probs = (0.6 * xgb_prob_smote) + (0.4 * lgb_prob_smote)
    report_metrics('Weighted Blend Ensemble (SMOTE)', y_valid, blend_probs)
    best_prob = blend_probs
elif xgb_prob_smote is not None:
    best_prob = xgb_prob_smote
elif lgb_prob_smote is not None:
    best_prob = lgb_prob_smote
elif xgb_prob is not None:
    best_prob = xgb_prob
elif lgb_prob is not None:
    best_prob = lgb_prob
else:
    print('No model probabilities available yet.')
```

```
=== Weighted Blend Ensemble (SMOTE) Performance ===
```

```
PR-AUC:          0.9011
ROC-AUC:         0.9987
Macro F1-Score: 0.9187
Minority F1:     0.8387
Balanced Acc:    0.9057

Confusion Matrix:
[[1799    2]
 [   3   13]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9983    0.9989    0.9986      1801
           1     0.8667    0.8125    0.8387        16

    accuracy                         0.9972      1817
   macro avg     0.9325    0.9057    0.9187      1817
weighted avg     0.9972    0.9972    0.9972      1817

==============================
```

1.4 Optional: Threshold Tuning (PR Curve)

[11]:
```
if best_prob is not None:
    precision, recall, thresholds = precision_recall_curve(y_valid, best_prob)
    f1_scores = (2 * precision[:-1] * recall[:-1]) / (precision[:-1] + recall[:
 ↪-1] + 1e-12)
    best_idx = int(np.argmax(f1_scores))
    best_threshold = thresholds[best_idx]
    print(f'Best F1 threshold: {best_threshold:.4f} | F1: {f1_scores[best_idx]:.
 ↪4f}')
    report_metrics('Best-threshold evaluation', y_valid, best_prob,␣
 ↪threshold=best_threshold)
else:
    print('No probabilities available for threshold tuning.')
```

```
Best F1 threshold: 0.3251 | F1: 0.9091
=== Best-threshold evaluation Performance ===
PR-AUC:          0.9011
ROC-AUC:         0.9987
Macro F1-Score: 0.9541
Minority F1:     0.9091
Balanced Acc:    0.9682

Confusion Matrix:
[[1799    2]
```

```
 [   1   15]]

Detailed Report:
              precision    recall  f1-score   support

           0     0.9994    0.9989    0.9992      1801
           1     0.8824    0.9375    0.9091        16

    accuracy                         0.9983      1817
   macro avg     0.9409    0.9682    0.9541      1817
weighted avg     0.9984    0.9983    0.9984      1817

==============================
```